



PROGRAMMATION ORIENTÉE OBJET

Cours 5



SURCHARGE

La surcharge de méthode est un concept qui permet à une classe d'avoir plusieurs méthodes portant le même nom, si leurs listes d'arguments sont différentes. Il est similaire à la surcharge de constructeur en Java, qui permet à une classe d'avoir plus d'un constructeur ayant des listes d'arguments différentes.

Conditions :

- ✓ Même nom de méthode,
- ✓ Paramètres différents (soit sur le nombre ou le/les type(s)),
- ✓ Le type de retour n'est pas pris en compte.

Cas valide de surcharge

```
somme(int, int)  
somme(int, int, int)
```

```
somme(int, int)  
somme(int, double)
```

Cas non valide de surcharge

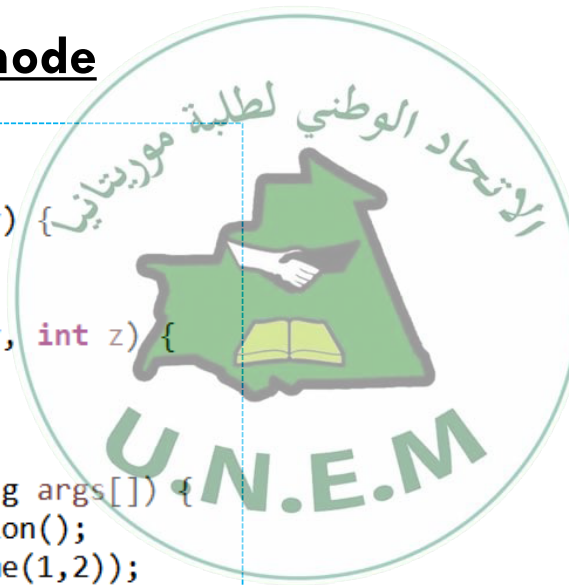
```
int somme(int, int)  
double somme(int, int)
```



SURCHARGE

Exemple : Surcharge de méthode

```
public class Operation {  
  
    public int somme(int x, int y) {  
        return x + y;  
    }  
  
    public int somme(int x, int y, int z) {  
        return x + y + z;  
    }  
  
    public static void main(String args[]) {  
        Operation op = new Operation();  
        System.out.println(op.somme(1,2));  
        System.out.println(op.somme(1,2,3));  
    }  
}
```



SURCHARGE

Exemple : Surcharge de constructeur

```
public class Operation {  
  
    int x, y;  
  
    public Operation() {  
  
    }  
  
    public Operation(int y) {  
        this.x = 0;  
        this.y = y;  
    }  
  
    public Operation(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public int somme(int x, int y) {  
        return x + y;  
    }  
  
    public static void main(String args[]) {  
        Operation op = new Operation(3);  
        System.out.println(op.somme(op.x, op.y));  
    }  
}
```

```
public class Operation {  
  
    int x, y;  
  
    public Operation() {  
  
    }  
  
    public Operation(int y) {  
        this.x = 0;  
        this.y = y;  
    }  
  
    public Operation(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public int somme(int x, int y) {  
        return x + y;  
    }  
  
    public static void main(String args[]) {  
        Operation op = new Operation(3,4);  
        System.out.println(op.somme(op.x, op.y));  
    }  
}
```



HÉRITAGE

Définition :

C'est un concept en POO qui désigne le fait qu'une classe peut hériter des caractéristiques (attributs et méthodes) d'une autre classe.

Avantages :

Spécialisation, enrichissement : une nouvelle classe réutilise les attributs et les opérations d'une classe en y ajoutant et/ou des opérations particulières à la nouvelle classe

Redéfinition : une nouvelle classe redéfinit les attributs et opérations d'une classe de manière à en changer le sens et/ou le comportement pour le cas particulier défini par la nouvelle classe

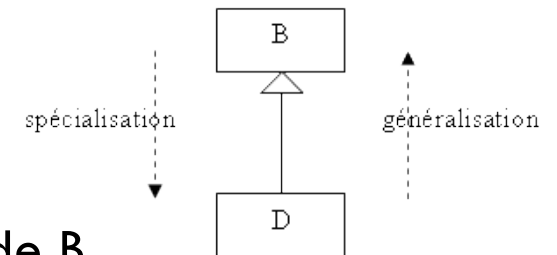
Réutilisation : évite de réécrire du code existant et parfois on ne possède pas les sources de la classe à hériter



HÉRITAGE

Soient deux classes D et B

- ✓ Relation d'héritage : Classe D « étend la » Classe B
- ✓ B est la **super-classe** ou classe mère/parent de D
- ✓ D est la **sous-classe**, la classe fille ou la classe dérivée de B



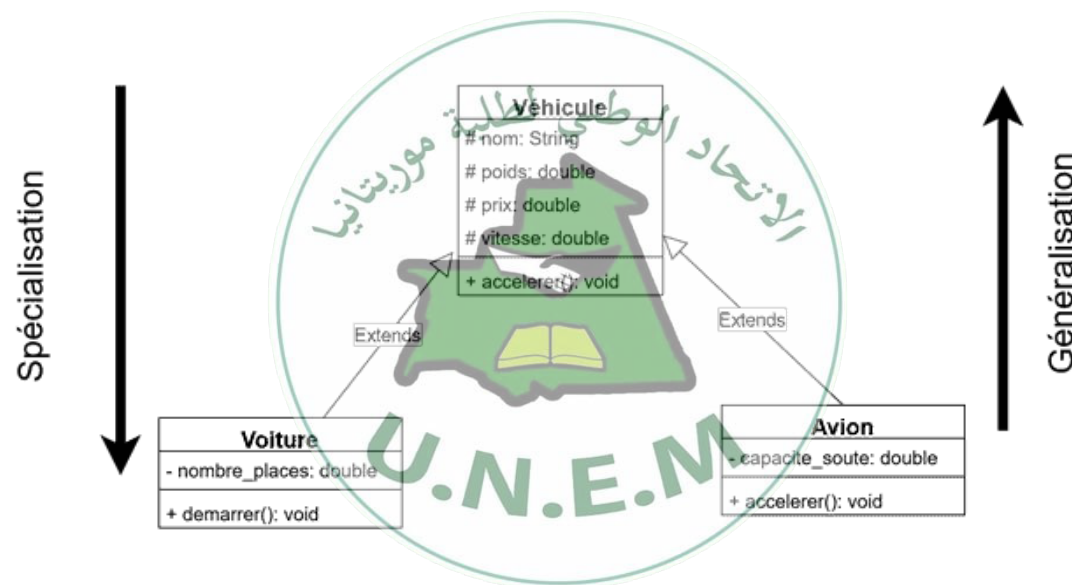
Syntaxe Java :

Utilisation du mot-clé **extends** après le nom de la classe

```
public class D extends B
{
}
}
```



HÉRITAGE

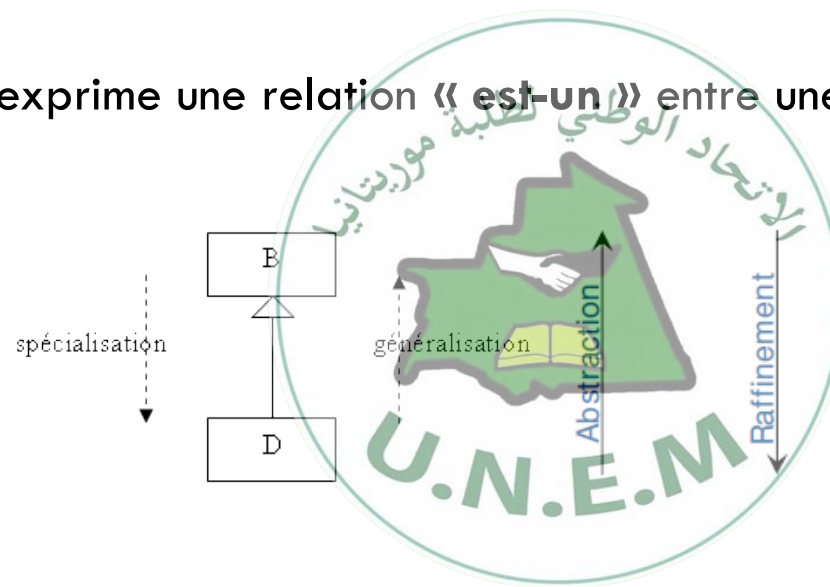


Dans l'exemple ci-dessus la classe **Voiture** et **Avion** sont des **classes filles** de la classe **Vehicule**. On peut les concevoir comme étant des **classes spécialisées** de la classe **Vehicule**. Ainsi tout objet de la classe **Voiture** ou **Avion** possédera les attributs et méthodes de la classe **Vehicule** en plus de ses propres caractéristiques.



HÉRITAGE

La généralisation exprime une relation « est-un » entre une **classe** et sa **super-classe**



L'héritage permet :

- ✓ de **généraliser** dans le sens abstraction
- de **spécialiser** dans le sens raffinement



HÉRITAGE

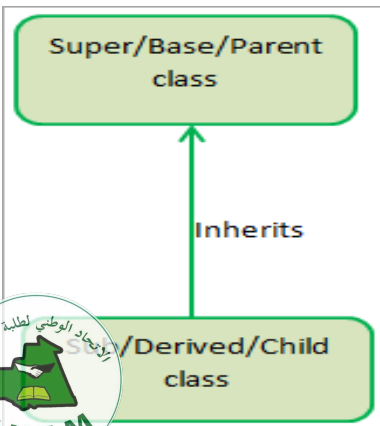
Exemple



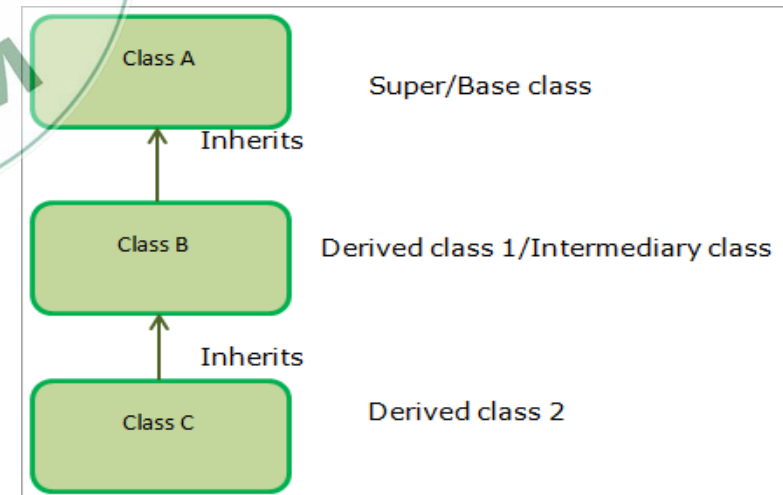
HÉRITAGE

Types d'héritage:

1) **Héritage unique (simple):** Lorsqu'une classe ou sous-classe dérivée hérite d'une seule base ou superclasse, il s'agit d'un héritage unique.



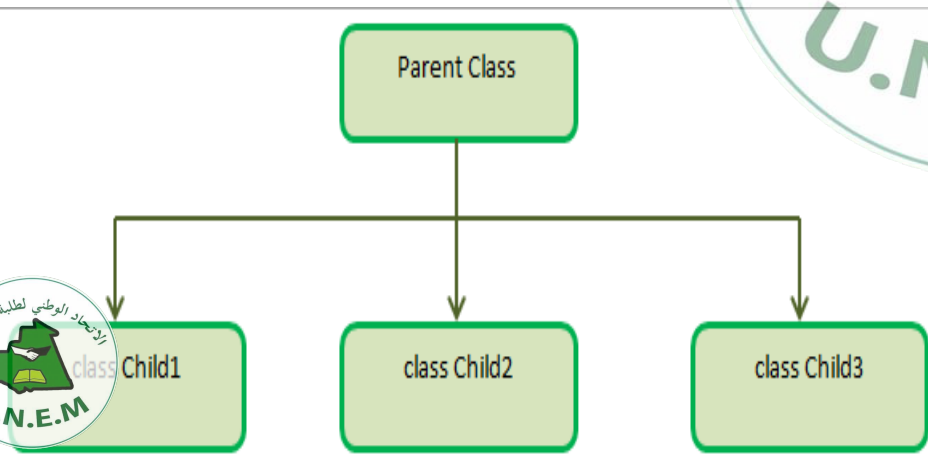
2) **Héritage à plusieurs niveaux:** Dans l'héritage à plusieurs niveaux, nous avons plus d'un niveau dans lequel une classe hérite d'une classe de base et la classe dérivée à son tour est héritée par une autre classe.



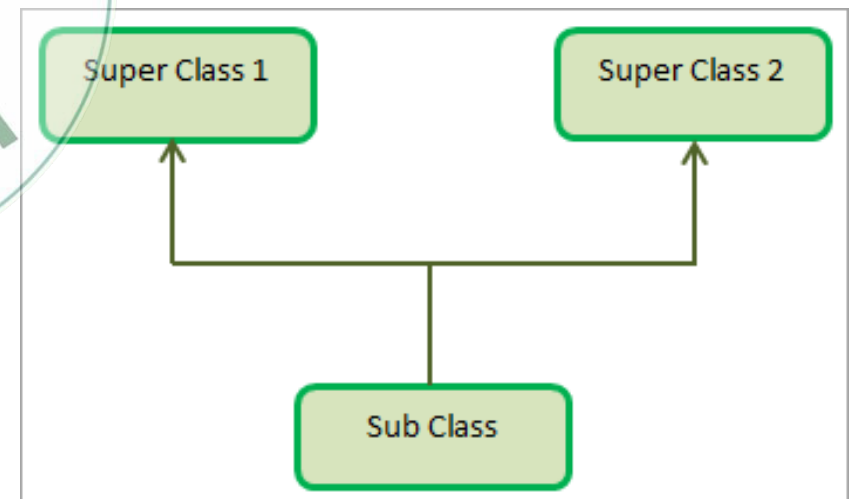
HÉRITAGE

Types d'héritage:

3) **Héritage hiérarchique:** Une hiérarchie d'héritage est formée dans ce type d'héritage lorsqu'une superclasse est héritée par plus d'une classe.



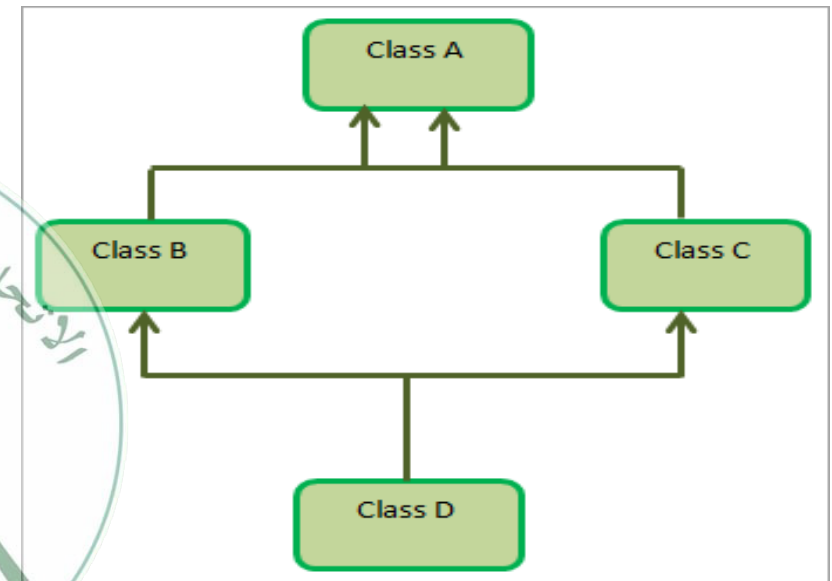
4) **Héritage multiple:** L'héritage multiple est celui dans lequel une classe peut hériter des propriétés et du comportement de plusieurs parents.



HÉRITAGE

Types d'héritage:

5) **Héritage hybride:** Lorsqu'un ou plusieurs types d'héritage sont combinés, cela devient un héritage hybride.



N.B:

- ✓ Java ne prend en charge que les types d'héritage à un, plusieurs niveaux et hiérarchiques utilisant des classes. Java ne prend pas en charge l'**héritage multiple** et **hybride** avec les **classes**.

Dans certains autres langages (ex : C++) : l'**héritage multiple** est possible



HÉRITAGE

La classe Object

La classe **Object** est la classe de plus haut niveau dans la hiérarchie d'héritage

- ✓ Toute classe autre que Object possède une **super-classe**
- ✓ Toute classe hérite directement ou indirectement de la classe **Object**
- ✓ Une classe qui ne définit pas de clause **extends** hérite de la classe **Object**

N.B: Il n'est pas nécessaire d'écrire explicitement **extends Object**

Object

```
+ Class getClass()  
+ String toString()  
+ boolean equals(Object)  
+ int hashCode()  
...
```



HÉRITAGE

Surcharge et redéfinition

- ✓ Une sous-classe peut ajouter des nouveaux attributs et/ou méthodes à ceux qu'elle hérite (**surcharge en fait partie**)
- ✓ Une sous-classe peut redéfinir (**redéfinition**) les méthodes à ceux dont elle hérite et fournir des implémentations spécifiques pour celles-ci

- ❖ **Rappel de la surcharge** : possibilité de définir des méthodes possédant le même nom mais dont les arguments (paramètres et valeur de retour) différent
- ❖ **Redéfinition (overriding)** : lorsque la sous-classe définit une méthode dont le nom, les paramètres et le type de retour sont identiques

Ne pas confondre surcharge et redéfinition



Dans le cas de la surcharge la sous-classe ajoute des méthodes tandis que la redéfinition « spécialise » méthodes existantes

HÉRITAGE

Surcharge et redéfinition

Sans redéfinition de la méthode m

```
class B {  
    void m() {  
        System.out.println("méthode m de B");  
    }  
}  
  
class D extends B{  
}
```

```
B b = new B();  
new D();
```

Résultat

```
méthode m de B  
méthode m de B
```

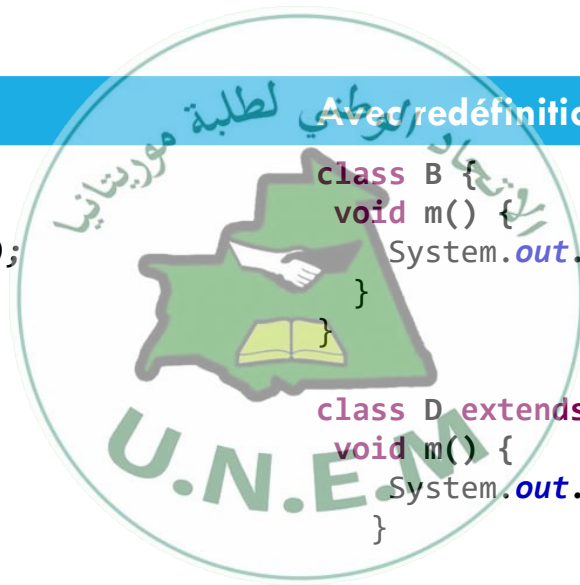
Avec redéfinition de la méthode m

```
class B {  
    void m() {  
        System.out.println("méthode m de B");  
    }  
}  
  
class D extends B{  
    void m() {  
        System.out.println("méthode m de D");  
    }  
}
```

```
B b = new B();  
D d = new D();  
b.m();  
b = d;  
b.m();
```

Résultat

```
méthode m de B  
méthode m de D
```



HÉRITAGE

Surcharge et redéfinition

Intérêt :

- ✓ La **redéfinition** d'une méthode cache le code de la **méthode héritée**
- ✓ **Réutiliser** le code de la méthode hérité par le **mot-clé super**

Le mot-clé super :

Le mot-clé **super** en java est une variable de référence qui est utilisée pour référencer les objets de la **classe mère**. Le mot-clé **super** est principalement utilisé dans les contextes suivants:

- ✓ Pour référencer une **variable d'instance** de la **classe mère**.
- ✓ Pour appeler la **méthode** de la **classe mère** immédiate.
- ✓ Pour appeler le **constructeur** de la **classe mère** immédiat



HÉRITAGE

this et super:

Chaque instance est munie de deux références particulières :

- ✓ **this** réfère l'instance elle-même.
- ✓ **super** réfère la partie héritée de l'instance.



HÉRITAGE

1) Utilisation de **super** avec une variable d'instance:

```
class A{
    String name = "A";
}

class B extends A {
    String name = "B";
    void showName(){
        //affiche le nom de la classe B
        System.out.println(name);
        //affiche le nom de la classe A
        System.out.println(super.name);
    }
}
```

```
class Main {
    public static void main(String args[]){
        B b = new B();
        b.showName();
    }
}
```

2) Utilisation de **super** avec des méthodes:

```
class A{
    void show(){ System.out.println("Ceci est la classe A."); }
}

class B extends A {
    void show(){ System.out.println("Ceci est la classe B."); }
    void display(){
        show();
        super.show();
    }
}
```

```
class Main {
    public static void main(String args[]){
        B b = new B();
        b.display();
    }
}
```

HÉRITAGE

3) Utilisation de **super** avec des constructeurs :

```
class A{
    A(){ System.out.println("A est créé."); }
}

class B extends A {
    B(){
        super();
        System.out.println("B est créé.");
    }
}

class Main {
    public static void main(String args[]){
        B b = new B();
    }
}
```



HÉRITAGE

Usage des constructeurs

- ✓ Possibilité comme les méthodes de réutiliser le code des constructeurs de la **super-classe**
- ✓ Appel explicite d'un constructeur de la **classe mère** à l'intérieur d'un constructeur de la **classe fille**
 - Utilise le mot-clé **super** : **super(paramètres du constructeur);**
- ✓ Appel implicite d'un constructeur de la classe mère est effectué quand il n'existe pas d'appel explicite. Java insère implicitement l'appel **super()**
- ✓ L'appel au constructeur de la **superclasse** doit se faire absolument en **première instruction**

HÉRITAGE

```
public class A {  
    • public A() {  
        System.out.println("Classe A");  
    }  
}
```

```
public class B extends A {  
    • public B(String message) {  
        super(); // Appel implicite  
        System.out.println("Classe B");  
        System.out.println(message);  
    }  
}
```

```
public class C extends B {  
    • public C(String debut) {  
        super("Message issu C" + debut);  
        System.out.println("Classe C");  
        System.out.println("Fin");  
    }  
}
```

```
public class Test {  
    public static void main (String[] argv) {  
        new C(" et Message du main");  
    }  
}
```

Exemple d'usage de
constructeurs avec
Héritage



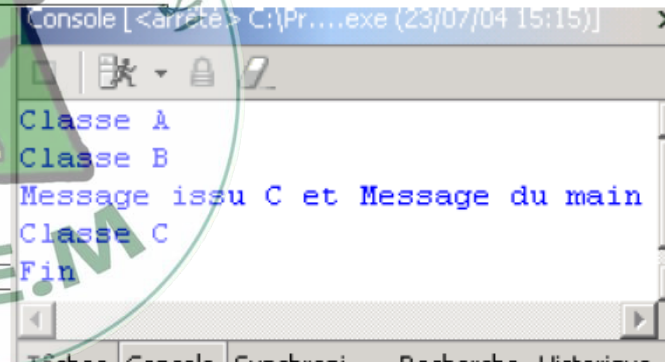
HÉRITAGE

```
public class A {  
    • public A() {  
        System.out.println("Classe A");  
    }  
}
```

```
public class B extends A {  
    • public B(String message) {  
        super(); // Appel implicite  
        System.out.println("Classe B");  
        System.out.println(message);  
    }  
}
```

```
public class C extends B {  
    • public C(String debut) {  
        super("Message issu C" + debut);  
        System.out.println("Classe C");  
        System.out.println("Fin");  
    }  
}
```

```
public class Test {  
    public static void main (String[] argv) {  
        new C(" et Message du main");  
    }  
}
```



```
Console [ <arrête> C:\Pr...exe (23/07/04 15:15)]  
Classe A  
Classe B  
Message issu C et Message du main  
Classe C  
Fin
```



HÉRITAGE

```
class X{  
    int x;  
  
    public X (int x){  
        this.x = x;  
    }  
}
```

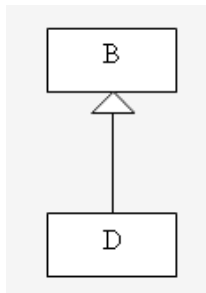


Erreur de compilation : le super constructeur implicite X() n'est pas défini pour le constructeur par défaut.



HÉRITAGE

Droits d'accès aux attributs et méthodes



```
class B {
    private int a;
    protected int b;
    int c;
    public int d;
}

class D extends B {
    int x;
    void f() {
        b = 5; OK
        d = 3; OK
        a = 2; NO
    }
}
```

Seulement les attributs et membres déclarés avec des modificateurs **protected** et **public** qui sont accessibles à partir de la **classe fille**